ПРАКТИЧНЕ ЗАНЯТТЯ №9

Тема: Оброблення виняткових ситуацій

Виконав: Вівчар Вадим Вікторович

Група: АЛК-43

Мета роботи

Ознайомитися з механізмом оброблення виняткових ситуацій у мові C++, розглянути призначення операторів try, catch і throw, а також з’ясувати, як здійснюється перехоплення, передавання та загальна обробка винятків.

Теоретичні відомості

Виняткова ситуація — це подія, що виникає під час виконання програми і вимагає спеціальної обробки. У мові C++ виняткова ситуація може бути представлена об’єктом будь-якого типу: числом, рядком, об’єктом класу тощо. Механізм оброблення винятків побудований на трьох ключових словах: try, catch і throw. Блок try містить код, у якому може виникнути помилка; оператор throw генерує виняткову ситуацію; блок catch перехоплює її та визначає подальшу реакцію програми.

Якщо під час виконання блоку try виняток не виникає, усі блоки catch ігноруються. Якщо ж виняток згенеровано, подальше виконання операторів у поточному блоці try припиняється, а керування передається до першого відповідного блоку catch. Якщо відповідного catch не знайдено, програма завершується аварійно.

1. Що таке виняткова ситуація?

Виняткова ситуація — це будь-яка подія під час виконання програми, що вимагає особливої обробки. Це може бути як помилка програмування, так і небажана, але передбачувана ситуація, наприклад спроба ділення на нуль, вихід за межі масиву або порушення певної умови в роботі програми.

2. Запишіть загальні форми операторів try, catch і throw. Опишіть їх функції.

Загальна форма має такий вигляд: блок try містить фрагмент коду, де може виникнути помилка; оператор throw використовується для генерації виняткової ситуації; один або декілька блоків catch перехоплюють виняток певного типу. Функції цих операторів такі: try — захищає код; throw — передає виняткову ситуацію; catch — приймає її та виконує обробку.

3. Як сумісна робота операторів try, catch і throw забезпечує в C++ оброблення виняткових ситуацій?

Програміст розміщує потенційно небезпечний код у блоці try. Якщо під час виконання виявляється умова, яку слід трактувати як помилку, генерується виняток оператором throw. Після цього керування негайно переходить до відповідного блоку catch, аргумент якого сумісний з типом згенерованого винятку. Таким чином, виконання програми не обов’язково завершується аварійно: замість цього можна вивести повідомлення, виправити ситуацію або коректно завершити роботу.

4. Чи можна використовувати throw, якщо виконання програми не проходить через блок try?

Так, оператор throw може бути використаний і поза блоком try, наприклад усередині окремої функції. Проте для коректної обробки виняток має бути перехоплений десь вище по ланцюжку викликів. Якщо жодного відповідного блока catch не буде знайдено, програма завершиться аварійно.

5. Яка форма оператора catch буде обробляти всі типи виняткових ситуацій?

Для перехоплення будь-яких винятків використовується універсальна форма catch(...). Такий блок не має іменованого параметра і підходить для будь-якого типу виняткової ситуації. Його доцільно розміщувати останнім після більш конкретних блоків catch.

6. Яким буде результат генерації виняткової ситуації, для якої не задано відповідного оператора catch?

Якщо для згенерованого винятку не знайдено відповідного блока catch, відбувається виклик стандартного механізму завершення програми. У такому разі викликається стандартна функція terminate(), яка приводить до аварійного завершення роботи програми.

Приклади використання

Нижче наведено короткі ілюстративні приклади, що відображають основні випадки, розглянуті в лекції: просте перехоплення винятку, передавання винятку з функції та універсальне перехоплення через catch(...).

Приклад 1. Проста обробка виняткової ситуації

У цьому прикладі виняток генерується безпосередньо в блоці try. Якщо знаменник дорівнює нулю, оператор throw передає рядок у відповідний catch, після чого програма виводить повідомлення і продовжує коректне завершення.

```
#include <iostream>
```

```
using namespace std;

int main()
{
    int a = 10;
    int b = 0;

    try
    {
        if (b == 0)
            throw "Division by zero";

        int result = a / b;
        cout << "Result: " << result << endl;
    }
    catch (const char* error)
    {
        cout << "Error: " << error << endl;
    }

    cout << "Program finished." << endl;
    return 0;
}
```

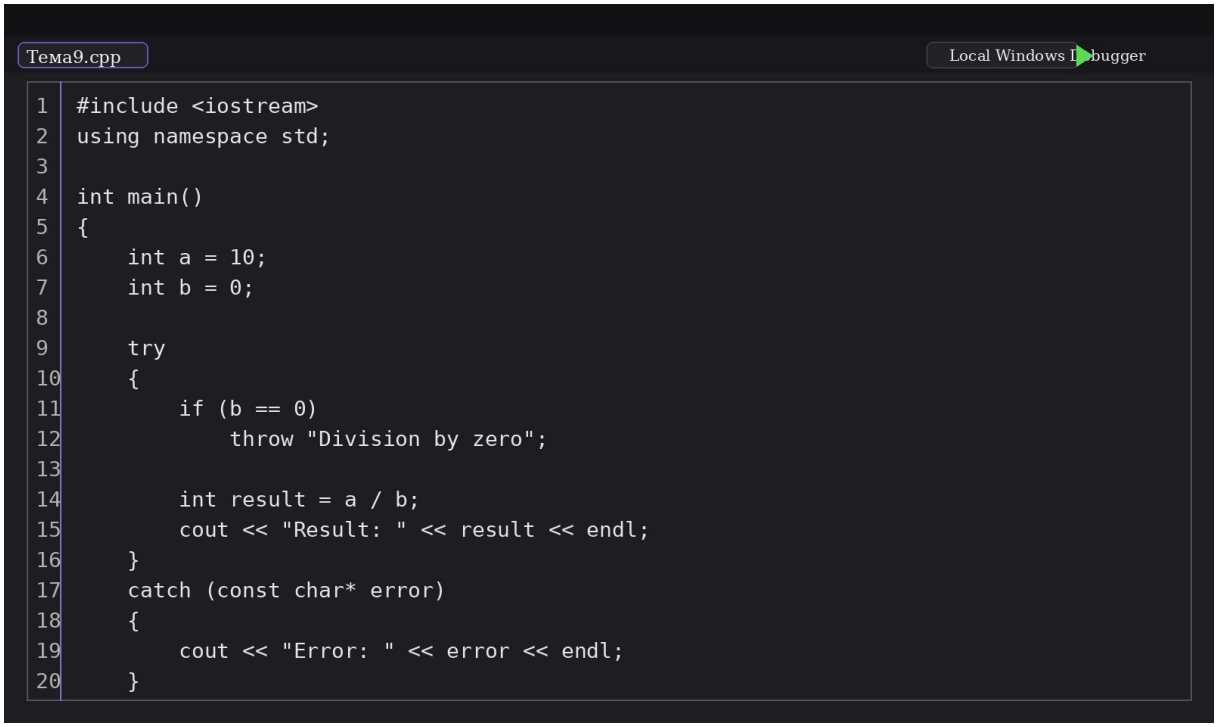

Рисунок 1 – Код програми для оброблення ділення на нуль у Visual Studio

```
Microsoft Visual Studio Debug Console

Error: Division by zero
Program finished.

E:\user vadim\Don\Tema9\x64\Debug\Tema9.exe (process 12712) exited with code 0 (0x0).
Press any key to close this window . . .
```

Рисунок 2 – Результат виконання програми з перехопленням винятку ділення на нуль

Приклад 2. Генерація винятку всередині функції

Тут оператор throw знаходиться всередині окремої функції. Виняток виникає не в main(), але передається вгору по ланцюжку викликів і перехоплюється блоком catch у головній функції. Такий підхід зручний, коли функція лише виявляє помилку, а її обробка виконується на вищому рівні.

```
#include <iostream>
using namespace std;

int getDenominator(int value)
{
    if (value == 0)
        throw "Division by zero in function";

    return value;
}

int main()
{
    try
    {
        int denominator = getDenominator(0);
        int result = 100 / denominator;
        cout << "Result: " << result << endl;
    }
    catch (const char* error)
```

```
    {
        cout << "Caught error: " << error << endl;
    }

    cout << "Program finished." << endl;
    return 0;
}
```

1.cpp

Local Windows Debugger

```
1  #include <iostream>
2  using namespace std;
3
4  int getDenominator(int value)
5  {
6      if (value == 0)
7          throw "Division by zero in function";
8
9      return value;
10 }
11
12 int main()
13 {
14     try
15     {
16         int denominator = getDenominator(0);
17         int result = 100 / denominator;
18         cout << "Result: " << result << endl;
19     }
20     catch (const char* error)
```

Рисунок 3 – Код програми з генерацією винятку у функції

Microsoft Visual Studio Debug Console

```
Caught error: Division by zero in function
Program finished.

E:\user vadim\Don\Tema9.1\x64\Debug\Tema9.1.exe (process 5944) exited with code 0 (0x0).
Press any key to close this window . . .
```

Рисунок 4 – Результат виконання програми з перехопленням винятку з функції

Приклад 3. Універсальне перехоплення через catch(...)

Форма catch(...) використовується тоді, коли потрібно підстрахувати програму і перехопити будь-який виняток, навіть якщо його точний тип не був передбачений заздалегідь. Зазвичай такий блок розміщують останнім у ланцюжку обробників.

```
#include <iostream>
using namespace std;

int main()
{
    try
    {
        throw 100;
    }
    catch (...)
    {
        cout << "Unknown exception was caught." << endl;
    }

    cout << "Program finished." << endl;
    return 0;
}
```

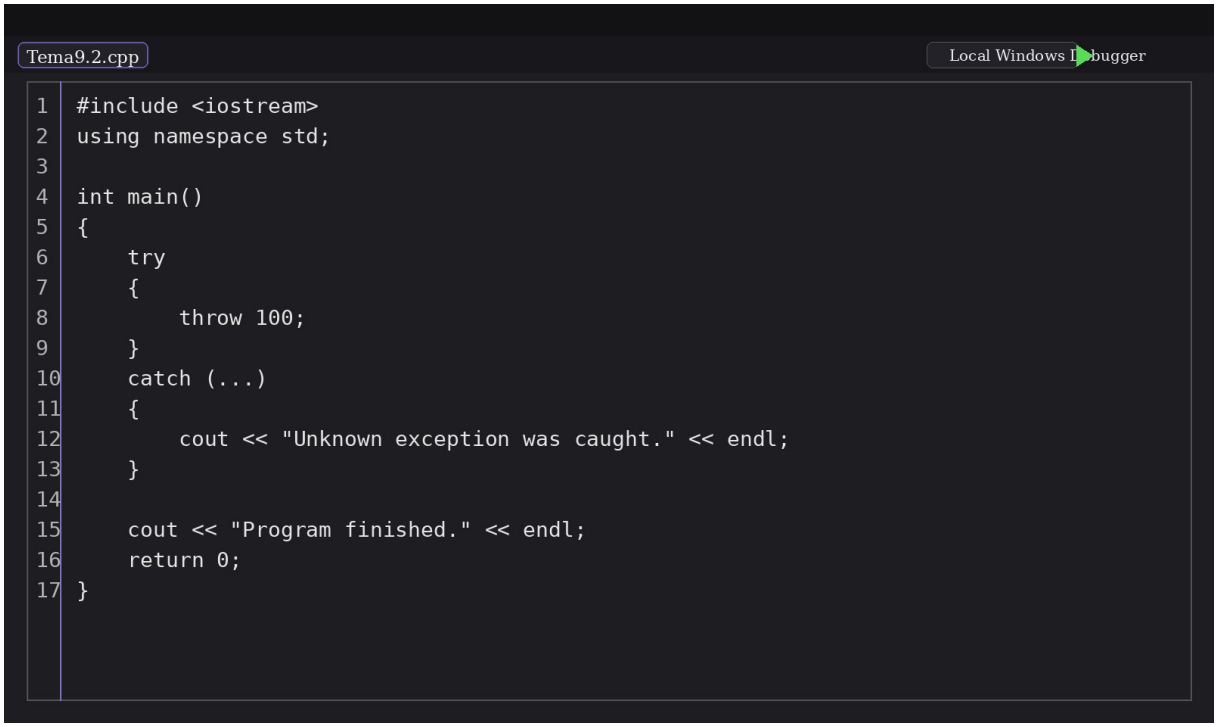

Рисунок 5 – Код програми з універсальним блоком catch(...)

```
Microsoft Visual Studio Debug Console

Unknown exception was caught.
Program finished.

E:\user vadim\Don\Tema9.2\x64\Debug\Tema9.2.exe (process 1972) exited with code 0 (0x0).
Press any key to close this window . . .
```

Рисунок 6 – Результат виконання програми з універсальним перехопленням винятку

Додаткові приклади, виконані у Visual Studio

Окрім основних прикладів, було виконано ще два варіанти програм, які демонструють оброблення різних типів винятків та перевірку умови з використанням оператора throw.

Приклад 4. Оброблення різних типів винятків

У цьому прикладі використано кілька блоків catch для різних типів винятків. Програма дозволяє окремо обробляти цілочисельний та текстовий винятки залежно від введеного користувачем значення.

```
#include <iostream>
using namespace std;

int main()
{
    try
    {
        int choice;
        cout << "Enter 1 for int exception, 2 for string exception: ";
        cin >> choice;

        if (choice == 1)
            throw 404;
        else if (choice == 2)
            throw "Text exception";
        else
```

```
            cout << "No exception." << endl;
    }
    catch (int errorCode)
    {
        cout << "Integer exception caught: " << errorCode << endl;
    }
    catch (const char* message)
    {
        cout << "String exception caught: " << message << endl;
    }

    cout << "Program finished." << endl;
    return 0;
}
```

Tama9.3.cpp

Local Windows Debugger

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      try
7      {
8          int choice;
9          cout << "Enter 1 for int exception, 2 for string exception: ";
10         cin >> choice;
11
12         if (choice == 1)
13             throw 404;
14         else if (choice == 2)
15             throw "Text exception";
16         else
17             cout << "No exception." << endl;
18     }
19     catch (int errorCode)
20     {
```

Рисунок 7 – Код програми з кількома блоками catch

```
Microsoft Visual Studio Debug Console

Enter 1 for int exception, 2 for string exception: 1
Integer exception caught: 404
Program finished.

E:\user vadim\Don\Tama9.3\x64\Debug\Tama9.3.exe (process 15808) exited with code 0 (0x0)
Press any key to close this window . . .
```

Рисунок 8 – Результат виконання програми з перехопленням цілочисельного винятку

Приклад 5. Перевірка умови з використанням throw

У п’ятому прикладі оператор throw використовується для контролю доступу за віком користувача. Якщо введене значення менше 18, генерується виняток, який перехоплюється у блоці catch; інакше програма повідомляє про успішний доступ.

```
#include <iostream>
using namespace std;

int main()
{
    int age;
    cout << "Enter your age: ";
    cin >> age;

    try
    {
        if (age < 18)
            throw "Access denied. Age is less than 18.";

        cout << "Access granted." << endl;
    }
    catch (const char* error)
    {
        cout << error << endl;
    }
```

```
    return 0;
}
```

Tema9.3.cpp Local Windows Debugger

```
#include <iostream>
using namespace std;

int main()
{
    int age;
    cout << "Enter your age: ";
    cin >> age;

    try
    {
        if (age < 18)
            throw "Access denied. Age is less than 18.";

        cout << "Access granted." << endl;
    }
    catch (const char* error)
    {
        cout << error << endl;
    }
```

Рисунок 9 – Код програми з перевіркою віку користувача

Microsoft Visual Studio Debug Console

```
Enter your age: 23
Access granted.

E:\user vadim\Don\Tema9.3\x64\Debug\Tema9.3.exe (process 5976) exited with code 0 (0x0).
Press any key to close this window . . .
```

Рисунок 10 – Результат виконання програми при коректному значенні віку

Висновок

У ході роботи було розглянуто основи механізму оброблення виняткових ситуацій у мові C++. З’ясовано, що виняткова ситуація є подією, яка потребує спеціальної реакції програми. Показано призначення

операторів try, catch і throw, а також принцип їхньої спільної роботи. Окремо встановлено, що оператор throw може використовуватися і в окремих функціях, а для універсального перехоплення застосовується конструкція catch(...). Якщо відповідний блок catch відсутній, програма завершується аварійно. Отже, механізм винятків є важливим засобом підвищення надійності та керованості програм на C++.